```
                """.executeUpdate().apply()
    }
}
```

❶ Interface to abstract any repository implementation

❷ Transaction

❸ Postgres upsert statement

You could forgo creating the interface in this example, but the code is short and it is always best practice to add this level of indirection. This way, you can test the projection without needing a database up and running. (You could simulate it by creating a `Map[String,Int]` inside the class and updating it instead of the SQL statement.) Most of this code is implementation details that are secondary to the fact that you need to add an implementation to the repository. You can use any means to store the event you are processing. In this case, you use `ScalikeJDBC` as the database access library. Although the details of using this library are beyond the scope of this book, let's clarify a few details.

`session.db.withinTx` executes the statement as a transaction and initiates a beginning with two possible outcomes: commit or rollback. `implicit dbSession =>` states that this transaction needs a variable of type `scalikejdbc.DBSession` on the scope, which means you have to define such an `implicit` variable before you can call this repository. You learn how to implement this with the `ScalikeJdbcSetup` later in this chapter.

10.3.3 Putting everything together

To create the projection, assemble the following:

- The `EventSourcedProvider` is the read side.
- The `JdbcProjection` with `CPCProjectionHandler` and `Cargos-PerContainerRepository` compose the write side.

Typically, you create a method like the following in your projection class:

```
object CargosPerContainerProjection {

  def createProjectionFor(       ❶
      system: ActorSystem[_],
      repository: CargosPerContainerRepository,
      indexTag: Int):
         ExactlyOnceProjection[Offset, EventEnvelope[SPContainer.Event]]

  ...
}
```

❶ Instantiates the projection

This is used in an application with an `ActorSystem`, so you have the `system`. Then `indexTag` creates the tags for querying the journal, as mentioned earlier.

The `createProjectionFor` method is coded in the `CargosPerContainerProjection` class. Here is the complete implementation previously shown partially in listing 10.5.

Listing 10.8 `CargosPerContainerProjection` implementation